

Singly Linked List Practice Worksheet

C++ Exercises from Basic to Intermediate

Name: _____

Date: _____

Quick Reminder

A singly linked list node usually looks like this:

```
struct Node {  
    int data;  
    Node* next;  
};
```

Each node stores a value in **data** and the address of the next node in **next**. The last node points to **NULL**.

1. Very Easy Exercises

1. Write the definition of a singly linked list node in C++.
2. Create one node with value 10 and set its **next** pointer to **NULL**.
3. Print the data of a single node.
4. Explain in one sentence what **head** means in a linked list.
5. Draw a linked list with 3 nodes containing: 5, 9, 12.

2. Traversal Practice

6. Write a function that prints all elements of a singly linked list.
7. Modify your printing function so that it prints arrows like 1 -> 2 -> 3 -> NULL.
8. Write a function that counts the number of nodes in a linked list.
9. Write a function that finds the sum of all node values.
10. Write a function that finds the maximum value in the list.

3. Insertion Exercises

11. Insert a new node at the beginning of the list.
12. Insert a new node at the end of the list.
13. Insert a new node after the first node.
14. Insert a new node at position 3 (1-based index).
15. Insert a node into an empty list.

4. Deletion Exercises

16. Delete the first node of a linked list.
17. Delete the last node of a linked list.

18. Delete the node at position 2.
19. Delete the first node only if the list is not empty.
20. Delete a node that contains a given value.

5. Searching and Updating

21. Search for a value in the linked list.
22. Count how many times a value appears in the list.
23. Update the first node that contains a specific value.
24. Replace every occurrence of 5 with 50.
25. Find the position of a value in the list.

6. Reverse and Middle Node

26. Reverse a linked list using iteration.
27. Reverse a linked list using recursion.
28. Find the middle node of a linked list.
29. Find the second middle node when the list has an even number of nodes.
30. Print the linked list in reverse order without changing the list.

7. Mixed Practice Problems

31. Build a linked list from user input until the user enters -1.
32. Display the list after every insertion.
33. Create a menu program with these options: insert at beginning, insert at end, delete first node, print, exit.
34. Sort a linked list in ascending order.
35. Merge two singly linked lists into one list.
36. Remove duplicate values from a linked list.
37. Check whether a linked list is empty.
38. Check whether the list contains exactly one node.
39. Find the length of the list without using recursion.
40. Free all allocated memory before the program ends.

8. Mini Project

Create a simple menu-driven program for a singly linked list with these features:

- insert at beginning
- insert at end
- delete first node

- delete last node
- search a value
- reverse list
- print list
- exit

Practice Checklist

Try to answer these questions after coding:

- Do I understand what `head` stores?
- Do I know how to move from one node to the next?
- Can I handle an empty list?
- Can I handle a list with only one node?
- Can I explain why `next = NULL` is important in the last node?

Challenge Question

Given this list:

$13 \rightarrow 11 \rightarrow 9 \rightarrow 7 \rightarrow 5 \rightarrow 3 \rightarrow 1 \rightarrow NULL$

Write a program to delete the first node and then print the updated list.

Good practice: write the code by hand first, then test it in your compiler.